

HERRAMIENTAS DE MAPEO OBJETO RELACIONAL (ORM)

Herramientas de Mapeo Objeto Relacional

El objetivo de esta Unidad es aprender a acceder a una base de datos relacional utilizando lenguaje orientado a objetos Java, en un contexto en el que disponemos de una interfaz que traduce la lógica de los objetos a la lógica relacional.

La **interfaz** se llama **ORM** (*Object Relational Mapping*) y es la herramienta que nos permitirá transformar representaciones de datos de los SGBDR a representaciones de objetos. Es decir =>

- ➡ **Las tablas serán clases**

- ➡ **Las filas o registros serán objetos**

Y se podrán manipular fácilmente como objetos.

Herramientas de Mapeo Objeto Relacional

El desfase Objeto-Relacional.

Los objetos dan solución a problemas complejos, y se utilizan cada vez más. Una BD relacional no está diseñada para almacenar objetos y al guardar los datos de un programa con el enfoque orientado a objetos se incrementa la complejidad de un programa, debido a la diferencia de esquemas entre los elementos a almacenar (objetos) y las características del repositorio de la BD (tablas). A esto se le denomina *desfase objeto-relacional*, y ocurre por la diferencia entre los modelos de datos de la BD y el lenguaje de POO.

Sin embargo, se desarrollan herramientas que proporcionan el “mapeo” de las estructuras de la BD y las utilizadas por el lenguaje.

Herramientas de Mapeo Objeto Relacional

Mapeo Objeto-Relacional (O/RM, ORM, O/R Mapping)

Es una técnica de programación para convertir datos entre el sistema de tipos utilizado en un lenguaje de programación orientado a objetos y el utilizado en una BD relacional, utilizando un motor de persistencia (se crea una BD orientada a objetos virtual).



Herramientas de Mapeo Objeto Relacional

Herramientas ORM. Características.

Las herramientas ORM nos permiten crear una capa de acceso a datos; una forma sencilla sería crear una clase por cada tabla de la BD y mapearlas una a una. Las herramientas son independientes de las BD utilizadas, de manera que permite migrar de una BD a otra sin tener que modificar el código (*sólo precisaría modificar lo relativo a la configuración*).

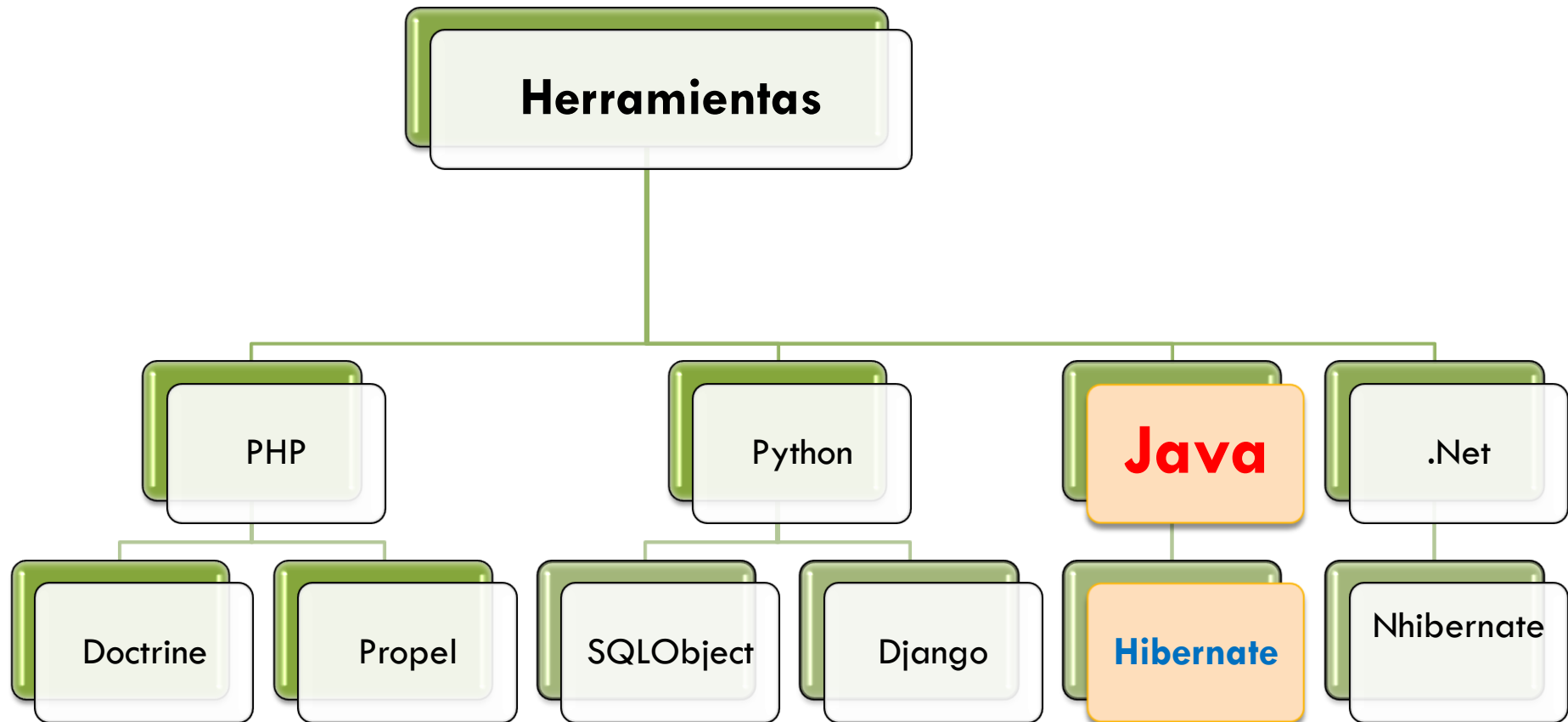
Ventajas

- Reducen el tiempo de desarrollo del sw
- Abstracción de la BD
- Reutilización
- Permiten la producción de mejor código
- Son independientes de la BD
- Lenguaje propio para realizar las consultas
- Propicia la portabilidad y la escalabilidad

Desventajas

- Son más lentas

Herramientas de Mapeo Objeto Relacional



Herramientas de Mapeo Objeto Relacional

HIBERNATE

Herramienta de mapeo objeto-relacional para la plataforma *Java*, mediante ficheros declarativos **XML**, que permite establecer las relaciones.

Proporciona una capa de persistencia que “abstrae” al programador *Java* de las particularidades de la BD. Aunque acepta la mayoría de sentencia SQL, no se utiliza sino que tiene su propio motor y haciendo uso de factorías (patrón de diseño *Factory*) y otros elementos de programación, construye las consultas, y proporciona el lenguaje **HQL** (*Hibernate Query Language*) para acceder a datos mediante POO.

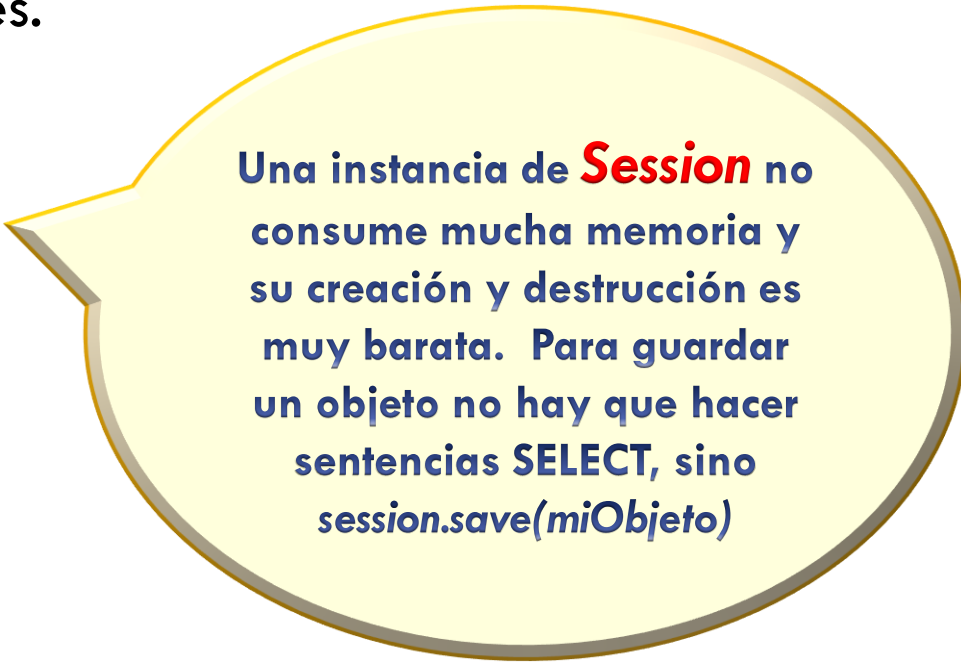
Herramientas de Mapeo Objeto Relacional

Arquitectura HIBERNATE.

El desarrollador mantiene una conversación con el motor de Hibernate mediante un objeto especial, la sesión (clase Session, equiparable al concepto de conexión de JDBC). Igualmente hay que crear y cerrar las sesiones.

Métodos de la clase Session:

- **save** (Object objeto)
- **createQuery** (String consulta)
- **beginTransaction()**
- **close()**



Una instancia de **Session** no consume mucha memoria y su creación y destrucción es muy barata. Para guardar un objeto no hay que hacer sentencias **SELECT**, sino `session.save(miObjeto)`

Herramientas de Mapeo Objeto Relacional

Interfaces de Hibernate:

- **SessionFactory:** permite obtener instancias Session.
- **Configuration:** para configurar Hibernate.
- **Query:** realiza consultas a la BD (HSQL o SQL nativo de la BD utilizada)
- **Transaction:** previene los errores.

El **Singleton** es un patrón de diseño para restringir la creación de objetos pertenecientes a una clase. El objetivo es garantizar que una clase solo tenga una instancia y proporcionar un punto de acceso global a ella. Se implementa creando en nuestra clase un método que crea una instancia del objeto solo si todavía no existen alguna. Para ello, se regula el alcance al constructor (con atributos como protegido o privado). Tienes un ejemplo => **HibernateUtil.java**

Herramientas de Mapeo Objeto Relacional

Instalación y Configuración de Hibernate:

- *Actividad guiada para la instalación, configuración y primer uso de Hibernate: [actividad 1_Instalacion y configuracion de Hibernate.docx](#)*

Herramientas de Mapeo Objeto Relacional

Estructura de los ficheros de Mapeo.

Los ficheros de mapeo son los que utiliza *Hibernate* para relacionar las tablas de la BD con los objetos Java. Están en formato xml y tienen la extensión .hbm.xml

```
<?xml version="1.0"?>  
<!DOCTYPE hibernate-mapping PUBLIC "-//Hibernate/Hibernate Mapping DTD 3.0//EN"  
"http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">  
<!-- Generated 25-nov-2014 23:16:20 by Hibernate Tools 3.4.0.CR1 -->  
<hibernate-mapping>
```

- 1) ***hibernate-mapping***: etiqueta de comienzo y final del fichero de mapeo.

Herramientas de Mapeo Objeto Relacional

2) **class**: etiqueta que engloba a la clase con sus atributos, en name se indica el nombre de la clase y en table se indica el nombre de la tabla a la que representa este objeto.

```
<class name="clases.Empleados" table="EMPLEADOS" schema="WORKRITA">
  <id name="numemp" type="big_decimal">
    <column name="NUMEMP" precision="22" scale="0" />
    <generator class="assigned" />
  </id>
  <many-to-one name="departamentos" class="clases.Departamentos" fetch="select">
    <column name="DEPTNO" precision="22" scale="0" />
  </many-to-one>
  <property name="nombre" type="string">
    <column name="NOMBRE" length="30" />
  </property>
```

Herramientas de Mapeo Objeto Relacional

Dentro de class distinguimos:

id, campo (***name***) que representa el atributo clave en la clase, con ***column*** que es su nombre en la tabla. Y ***type*** el tipo de datos. ***Generator*** indica la naturaleza del campo clave, *assigned* por usuario, *increment*, etc...

3) El resto de atributos se indican con la etiqueta ***property***, asociando el nombre del campo de la clase con el nombre de la columna de la tabla y el tipo de datos.

Los tipos que declaramos y utilizamos en los ficheros de mapeo no son tipos de datos Java, ni SQL. Se llaman **tipos de mapeo Hibernate**, convertidores que transforman tipos de datos Java a SQL y viceversa.

Herramientas de Mapeo Objeto Relacional

4) **many-to-one**, es una asociación unidireccional muchos a uno (clave ajena), en el ejemplo que estamos analizando, significa que muchos empleados pertenecen a un departamento. El mapeo quiere decir que la clase *Empleados.java* tiene un atributo de nombre *departamentos* que será una instancia de la clase *clases.Departamentos*, donde **fetch** por defecto es select y escoge entre la recuperación de unión exterior outer-join o la recuperación secuencial. Comprueba que en *Departamentos* la relación es *<one-to-many>*.

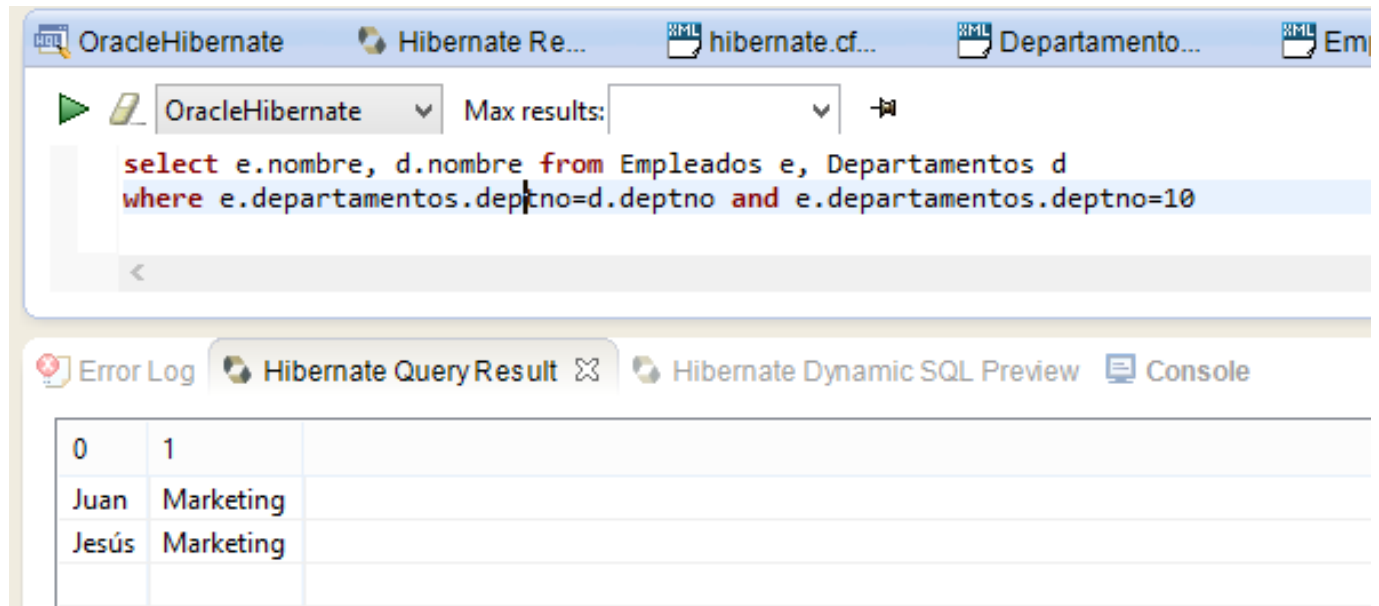
El atributo será *<set...>*, que indica que la clase *Departamentos.java* tiene un atributo de nombre **empleadoses** que es una **lista de instancias** de la clase *clases.Empleados*.

```
<set name="empleadoses" table="EMPLEADOS" inverse="true" lazy="true" fetch="select">
  <key>
    <column name="DEPTNO" precision="22" scale="0" />
  </key>
  <one-to-many class="clases.Empleados" />
</set>
```

Herramientas de Mapeo Objeto Relacional

¿Y cómo
implementamos la
relación de join en
una consulta HQL

El atributo de “*muchos*” que es clave foránea, es una instancia de la clase referenciada, por lo que hay que hacer referencia además, a la clase y al atributo de la siguiente manera:



The screenshot shows the OracleHibernate IDE interface. The top toolbar includes icons for running the query, saving, and other actions. The main text area contains the following HQL query:

```
select e.nombre, d.nombre from Empleados e, Departamentos d
where e.departamentos.deptno=d.deptno and e.departamentos.deptno=10
```

Below the query editor, the 'Hibernate Query Result' tab is active, displaying the results of the query in a table format:

0	1
Juan	Marketing
Jesús	Marketing

Herramientas de Mapeo Objeto Relacional

Clases persistentes.

A las clases generadas se les llama clases persistentes y equivalen a una tabla de la BD, y cada registro o fila es un objeto persistente de esa clase. En nuestro ejemplo las clases representan un objeto empleados y un objeto departamentos, por lo que podemos crear objetos empleados y departamentos a partir de ellas. Tendrán unos atributos y unos métodos *get* y *set* para acceder a ellos. Utilizan la notación de JavaBean.

Herramientas de Mapeo Objeto Relacional

Convenciones de nombrado estándares de JavaBean (*getters* y *setters*).

- Si la propiedad no es de tipo booleana, el nombre del método que lee el valor debe estar precedido por *get*: *getDnombre()*
- Si la propiedad es un booleano, el prefijo es *get* o *is*, por ej... sería *getJefe()* o *isJefe()*
- El método para almacenar un valor debe tener el prefijo *set*: *setDnombre()*
- Para completar el nombre de un método *getter* o *setter* solo hay que poner la primera letra que los une en mayúscula.
- Los métodos marcados como *setter* deben declararse como públicos, devolver un tipo void y recibir un argumento del tipo de propiedad al que van a dar valor.
- Los métodos de tipo *getter* deben ser declarados como públicos, no aceptan argumentos y devuelven un valor del mismo tipo que el que recibe el método *setter*.

Herramientas de Mapeo Objeto Relacional

Sesiones y Objetos Hibernate.

Para poder utilizar los mecanismos de persistencia de Hibernate se debe inicializar el entorno Hibernate y obtener un objeto Session utilizando la clase SessionFactory de Hibernate.

//Inicializa el entorno Hibernate

Configuration cfg=new Configuration().configure();

//carga el fichero de configuración hibernate.cfg.xml

//Crea la primera sessionfactory

SessionFactory sf=cfg.buildSessionFactory();

//se crea solo una vez

//Obtiene un objeto sesión

Session sesion=sf.openSession();

Herramientas de Mapeo Objeto Relacional

Carga, almacenamiento y manipulación de Objetos=>

Para recuperar una instancia persistente, tenemos el método **load()** de *Session*. Si el registro que se intenta recuperar no existe, se lanza una excepción irrecuperable, y para evitarlo se puede utilizar **get()**, que devuelve nulo si la fila no existe.

- ❖ *Hay algunas diferencias más, pero nos quedamos en que sólo usamos load() si realmente es un error grave el que no exista el registro.*

Para almacenar un objeto en la BD, tenemos el método **save()**.

Para realizar el borrado, **delete()**. Se requiere que previamente hayamos realizado el load() del objeto a eliminar.

Para modificarlo, utilizamos el método **update()** y al igual que con el borrado, hay que cargarlo previamente.

Herramientas de Mapeo Objeto Relacional

Parámetros de las consultas=> `createQuery()`

Método	Función
<code>setInteger(int posicion, int valor)</code> <code>setInteger(String nombre, int valor)</code>	Asigna valor a un parámetro de tipo entero
<code>setCharacter(int posicion, char valor)</code> <code>setCharacter(String nombre, char valor)</code>	Asigna valor a un parámetro de tipo char
<code>setDate(int posicion, Date fecha)</code> <code>setDate(String nombre, Date fecha)</code>	Asigna valor a un parámetro de tipo date
<code>setDouble(int posicion, double valor)</code> <code>setDouble(String nombre, double valor)</code>	Asigna valor a un parámetro de tipo decimal o float
<code>setString(int posicion, String valor)</code> <code>setString(String nombre, String valor)</code>	Asigna valor a un parámetro de tipo varchar
<code>setParameterList(String nombre, Collection valores)</code>	Asigna una lista de valores a un parámetro